Progress Report 2

Topic: Visual Drilling Operations & Predictive Maintenance System

Student Name: Uchechi Sebastian

Student ID: 300393092

Course: CSIS 4495 002 Applied Research Project

Section: 002

Team Lead: Uchechi Sebastian (Individual Project)

Table of Contents

Work Date / Hours Log **3**

Summary Description of work done during this reporting period 13

Repo Check in of Implementation completed **14**

- Proposal/ 14
- ProgressReports/ 14
- frontend/ 14
- backend/ 14
- data/ 15

AI Use Section **16**

AI use Table 16

Project Planning and Timeline **18**

Project Timeline 18

Gantt chat – Project timeline 19

Appendix **20**

AI Prompt(s) Used 20

Microsoft Copilot 21

Work Date / Hours Log

Date	Hours	Description
17-Jan-2026	3.5	- I explored and researched possible project ideas for the course, focusing on areas related to drilling operations and data analytics. I evaluated whether the project idea was feasible to implement within the course timeline and researched whether similar work had been done before. This helped me narrow down and decide on a suitable project that aligns with my background and course requirements.
20-Jan-2026	3	- I started learning React by reviewing the basics of components, JSX, and how a React application is structured. I focused on understanding how React will be used to build the frontend of my applied project.
21-Jan-2026	3.5	- I worked on Introduction of the project: Improving the introduction, stating the problem statement clearly, outlining what questions needs to be answered. - I searched for literature and research work based on my project to find similar research on my project. I searched for gaps within this research project work.
22-Jan-2026	2.5	- I continued literature research related to drilling operations, nonproductive time, and decision support systems. I reviewed how similar systems visualize operational data and noted limitations in existing approaches, particularly depth-based visualization. - I worked on my reference and included it in my project report after the research I did.
23-Jan-2026	3	- I worked on defining the project methodology, including data sources, data collection approach, and analysis techniques. - I also clarified how simulated drilling data would be used and documented the justification for using non-proprietary datasets in the project.
25-Jan-2026	1	- I created the project timeline for the complete project for the entire semester.

26-Jan-2026	3.5	- I drew the Gantt chart for the project timeline and included it in my proposal report. - I prepared the appendix. - I went through the entire report and did a lot of corrections and restructuring to make sure the proposal fits what my project is all about.
27-Jan-2026	2	- I set up the project GitHub repository structure according to the course requirements and organized folders for implementation and documentation. - I created the frontend, backend, and data directories under the implementation folder to support a clean development workflow.
29-Jan-2026	5	- I initialized the React frontend using Vite within the repository and verified that the development server runs correctly on my local machine. This step confirmed that the frontend environment was properly configured and ready for further development. - I continued learning React by implementing the initial application layout and navigation structure. I worked on creating reusable components for the application frame (layout and navigation) and began organizing the project into pages and components to follow best practices. I separated the HTML structure (JSX) and styling (CSS) into different files to improve readability and maintainability. I also installed and configured React Router to support page navigation between the wells list, dashboard, and reports sections of the application. - I implemented the well selection page and routing logic to allow navigation between different wells and their corresponding dashboard views. I tested navigation behavior to ensure that selecting a well correctly loads the appropriate dashboard page. During this process, I encountered and resolved common React

		development issues, including file naming inconsistencies, incorrect import paths, and component casing errors. I fixed these issues by standardizing file names, correcting relative import paths, and restarting the development server as needed, ensuring the application builds and runs successfully after all fixes.
02-Feb-2026	3.5	- Built and refined Wells page with color-coded well status cards - Implemented depth-based wellbore visualization - Added clickable depth segments with event details modal - Integrated routing between wells and dashboards - Improved frontend layout and interaction flow
05-Feb-2026	4	- I set up flexible column normalization so the upload can handle different Excel header names - I added validation for wells, operations, and events during the upload process - I added row-level error reporting to make debugging easier and catch bad data - I improved the duplicate checks so existing records aren't inserted - I added logging across the upload flow for better visibility - I finished wiring up the /upload endpoint with multipart file support - I updated the utils for mapping, normalization, and safer data parsing - I prepared the backend for future analytics by making sure uploaded data is clean and consistent

06-Feb-2026	2	- I created simulated datasets and collected sample datasets from different companies so I could compare how each one structures its drilling data. - I analyzed the differences in column names, formats, units, timestamps, and category labels across these datasets.
09-Feb-2026	5	- Implemented a complete rewrite of the Excel upload pipeline for wells, operations, and events. - Fixed all column-name mismatches between the transformer output and SQLAlchemy models. - Updated the Well insertion logic to use well_name instead of the invalid name argument. - Corrected the operations insertion to use operation_type instead of the non-existent op_type. - Updated event insertion and duplicate-check logic to use the correct model field (event_time). - I rebuilt the transform_dataset() function to ensure consistent normalization, depth conversion, timestamp parsing, and category mapping. - Ensured all transformed sheets contain the required columns before insertion. - Improved error reporting for wells, operations, and events to make debugging easier. - Cleaned up duplicate imports and removed unused or inconsistent logic. - Stabilized the entire ETL flow so uploads now run end-to-end with predictable behavior. -

13-Feb-2026	5	- Created operations router - Added GET /wells/{well_id}/operations endpoint - Added GET /wells/{well_id}/segments endpoint - Converted database operations into frontend-ready wellbore segments - Implemented basic risk-level classification logic - Connected router to main.py - Implemented DailyReport model and ingestion workflow for PDF uploads - Added duplicate detection using file hash and (well_id + report_date) - Created NNPC_FORMAT_A parser using pdfplumber with table-based extraction - Extracted operation rows from Operation Summary section - Implemented depth extraction logic (MD from / MD to detection) - Linked parsed operations to DailyReport and Well - Added debugging preview fields (matched_rows_preview, debug_preview). - Backend now supports: - Creating wells - Uploading daily drilling PDF reports - Preventing duplicate uploads - Parsing operation summary rows into database - Retrieving operations via GET endpoint
14-Feb-2026	4	- Added CORS + created /wells/{well_id}/dashboard endpoint and updated React Dashboards.jsx to fetch real segments/KPIs instead of hardcoded data. - Cleaned upload router to only handle PDF ingestion and moved wells/operations/segments endpoints under /wells routes to avoid duplicate Swagger entries. - Created wells router with create and list endpoints. Cleaned upload router to handle only report uploads.

		- Added CORS + created /wells/{well_id}/dashboard endpoint and updated React Dashboards.jsx to fetch real segments/KPIs instead of hardcoded data. - Fetch wells list from backend /wells/ Clicking a well navigates to /wells/:wellId dashboard
15-Feb-2026	4.5	- Added /reports router with list and detail endpoints - Enabled fetching all uploaded reports from DB - Enabled viewing a single report with its parsed operations - Registered router in main.py - Backend now supports a full Reports Page UI - Re-enable KPI light tone styling via tone classes Convert wellbore into fixed-height scroll view Add depth cursor slider + merge contiguous normal segments for cleaner visualization. - Implemented Reports.jsx to fetch /reports from backend - Added ReportRow.jsx component for list UI - Added Reports.css styling - Enabled navigation to individual report pages - Frontend now displays all uploaded drilling reports dynamically
16-Feb-2026	4	- Implemented a dedicated UploadReport.jsx page with styled form for uploading PDF reports. - Added UploadReport route in App.jsx and integrated upload button at the well level. - Added modal-based Create Well form directly inside Wells.jsx for quick well creation. - Redesigned Report Detail page to use a professional table view instead of cards. - Added new CSS: UploadReport.css and ReportDetail.css for a clean UI experience. - I Connected all frontend pages with backend routes properly. - Prepared UI structure for full multi-well report categorization.
19-Feb-2026	6	- Updated upload_daily-report FastAPI endpoint to use Form(...)

		for well_id, report_date, and parser_type. This ensures the frontend can upload reports using multipart/form-data. - Fixed 422 validation errors and restored successful PDF ingestion. - Added DELETE /reports/{report_id} endpoint in FastAPI - Added delete button to each report card in Reports.jsx - Added confirmation dialog and optimistic UI update after deletion - Added new CSS styles for delete button and improved card layout - Ensured seamless integration with existing upload and report detail flows. - fix(reports): register DELETE endpoint correctly - Moved delete_report() outside get_report_details() - Ensures FastAPI registers DELETE /reports/{report_id} - Resolves 405 Method Not Allowed during report deletion - main(wells): add Recently Viewed wells + Quick Actions bar + UI improvements - Added Quick Actions bar (Create Well, View Reports, Upload Report) - Added stats bar and modernized layout - Added color-coded well cards, operator avatars, and status badges - Improved overall dashboard experience - fix(wells): correct quick-action routing and add well-selection modal - View Reports and Upload Report now open a modal to choose a well - Added divider line to separate quick actions from wells - Improved navigation logic and UX consistency - style(wells): redesign Create Well modal for improved UX

		- Added modern modal card with animations - Improved input styling and labels - Added structured footer with Create + Cancel buttons - Enhanced spacing, shadows, and visual hierarchy
20-Feb-2026	4	- implement Sign Up page and full localStorage-based authentication - Added SignUp.jsx and SignUp.css - Updated Login.jsx to validate against stored user accounts - Added /signup route and integrated with login flow - Enabled full sign-up → login → dashboard experienc - added Sign Up link to Login page and complete auth flow - Added "Don't have an account? Create one" link - Linked Login → SignUp route - Updated styling for login switch link - Ensured smooth SignUp → Login → Wells navigation - Moved login/signup routes outside Layout wrapper - Added premium gradient background for auth pages - Optional glassmorphism card styling for modern UI
21-Feb-2026	2.5	- Replaced old icon with a cleaner well-bore-style SVG - Added new Home.css with modern gradient background and centered content - Updated Home.jsx to use new styling and improved structure - Ensured Home page loads without sidebar layout - Refined CTA button and feature pills for a more polished landing experience

23-Feb-2026	2	- Updated global App.css to ensure html, body, and #root all have width: 100% and height: 100% Prevented layout shrink issues caused by flex containers higher in the component tree Ensured that routed pages (e.g., Wells, Reports) now correctly expand to full available width Improved overall stability of the layout across all pages rendered inside <Layout/>
27-Feb-2026	4	- Fixed backend import error by correcting module paths in main.py(removed incorrect 'Implementation.' prefix) - Added predictive-maintenance API route with proper FastAPI router wiring - Introduced services/risk_engine.py for computing equipment risk scores - Ensured predictive.py imports use project-relative paths - Ensured predictive-maintenance results load dynamically in Maintenance.jsx - Cleaned router registration and removed duplicate predictive router include
4 -Mar-2026	6	**Dashboard & KPIs** • Fixed Event Count not updating by implementing proper cascade deletes. • Made all KPI cards clickable with short explanations. • Improved NPT KPI modal with a bar chart, proper labels, and correct report dates. **Report Detail** • Added editable mud and equipment tables under Operations, including mud descriptions. • Ensured Add/Edit buttons always show. • Resolved CORS/Vite proxy issues and added proper loading/error/editing states. **Backend** • Added mud_desc with automatic migration.

		• Added endpoints for updating equipment and adding mud. • Updated ingestion to store mud/equipment and return them in report details. **Parser (NNPC Format A)** • Extracted mud and equipment tables reliably using section-aware logic. • Prioritized tables with real data and expanded header keyword detection. **Wellbore Visualization** • Replaced fixed pipe height with dynamic height based on depth and segment count. • Removed CSS height constraints and prevented clipping by allowing overflow. **Equipment in Segment Modal** • Displayed equipment for each segment using equipmentByReport. • Passed equipment into SegmentModal and rendered a clean table when available. **AI Segment Analysis** • Added structured analysis output (reasons, factors, history, prevention, methodology). • Added endpoint for segment analysis and integrated it into SegmentModal with a full explanation view.

Summary Description of work done during this reporting period

This reporting period focused on expanding the platform into a fully functional multi-well drilling operations system by building out the core backend routers, ingestion workflows, parsing logic, and frontend integrations. I created dedicated routes for wells, operations, segments, and reports, and connected them cleanly into the application structure. I implemented the DailyReport model and the full PDF ingestion pipeline, including duplicate detection using file hashes and report metadata. I also developed the NNPC Format A parser using pdfplumber, enabling reliable extraction of operation summary rows, depth ranges, mud data, and equipment tables. Parsed data is now linked directly to wells and reports, with debugging previews available for validation.

On the backend, I added support for creating wells, uploading and parsing drilling reports, preventing duplicate uploads, and retrieving operations and wellbore segments through new GET endpoints. I introduced a dedicated dashboard endpoint, added CORS support, and reorganized routers to avoid Swagger duplication. I also built a complete reports router with list, detail, and delete functionality, fixed routing and import issues, and ensured the ingestion endpoint correctly handles multipart form data. These changes now allow the system to support a full Reports Page UI and seamless report deletion.

On the frontend, I connected all pages to live backend data. I implemented dynamic wells and reports pages, added navigation from wells to dashboards, and built a complete Reports interface with list views, detail pages, and deletion with confirmation. I created a dedicated Upload Report page and integrated upload actions at the well level. I added a modal-based Create Well form and redesigned the Report Detail page with a cleaner table layout. I also improved the wellbore visualization by converting it into a fixed-height scroll view, adding a depth cursor slider, and merging contiguous normal segments for clarity.

Beyond the core workflows, I modernized the Wells page with a Quick Actions bar, stats, color-coded well cards, operator avatars, and improved routing. I refined the Create Well modal with better styling and animations. I implemented a complete authentication flow with Sign Up, Login, and localStorage-based account handling, along with updated UI for both pages, including gradient backgrounds and a refreshed landing page. I also stabilized global layout behavior across routed pages to ensure consistent full-width rendering.

Finally, I added a predictive-maintenance API with a dedicated risk engine and integrated dynamic results into the Maintenance page. Altogether, I delivered an end-to-end system that

supports well creation, report ingestion, parsing, dashboard analytics, visualization, authentication, and predictive insights—backed by a cleaner UI and a more reliable backend architecture.

Repo Check in of Implementation completed

I have checked in my work to the GitHub repository according to the course requirements. The repository currently includes the following folders and files:

1. ReportsAndDocuments/
    - **Proposal/**
        - Project proposal document.
    - **ProgressReports/**
        - Progress Report #1
2. Implementation/
    - **frontend/**
        - Initialized React frontend project using Vite
        - src/ directory containing:
            - pages/
                - Wells
                - Dashboard
                - Reports
                - Maintenance
            - components/
                - KpiCard
                - KPIModal
                - Layout
                - ReportRow
                - SegmentModal
                - Wellbore
            - styles/ (separate CSS files for pages and components)
        - package.json and related configuration files
    - **backend/**
        - app/
            - __init__.py
            - main.py
        - models/
            - __init__.py
            - well.py
            - operation.py
            - event.py
        - routers/
            - __init__.py
            - wells.py

        - operations.py
        - events.py
        - analytics.py
        - risk.py
        - upload.py

    - schemas/
        - well_schema.py
        - operation_schema.py
        - event_schema.py
    - services/
        - injestion_service.py
        - ai_engine.py
        - kpi_services.py
        - risk_services.py

    - utils/
        - __init__.py
        - column_mapping.py
        - transform.py
        - database.py
- **data/**
    - Folder created to store simulated or example datasets for later phases

3. Miscellaneous - Misc
4. README.md

AI Use Section

AI use Table

AI Tool	Version / Account	Specific Use	Value Added
ChatGPT	GPT-5.2	Assisted with refining the project idea, structuring the proposal, improving academic writing, and clarifying the research design, methodology, and timeline.	Helped organize thoughts clearly, improve academic clarity, and ensure the project scope and structure align with the course.
Copilot	Free	Summary of Relevant Literature and Existing Research	Helped to clearly identify the existing research on the project and showed how the project differs from various working projects.

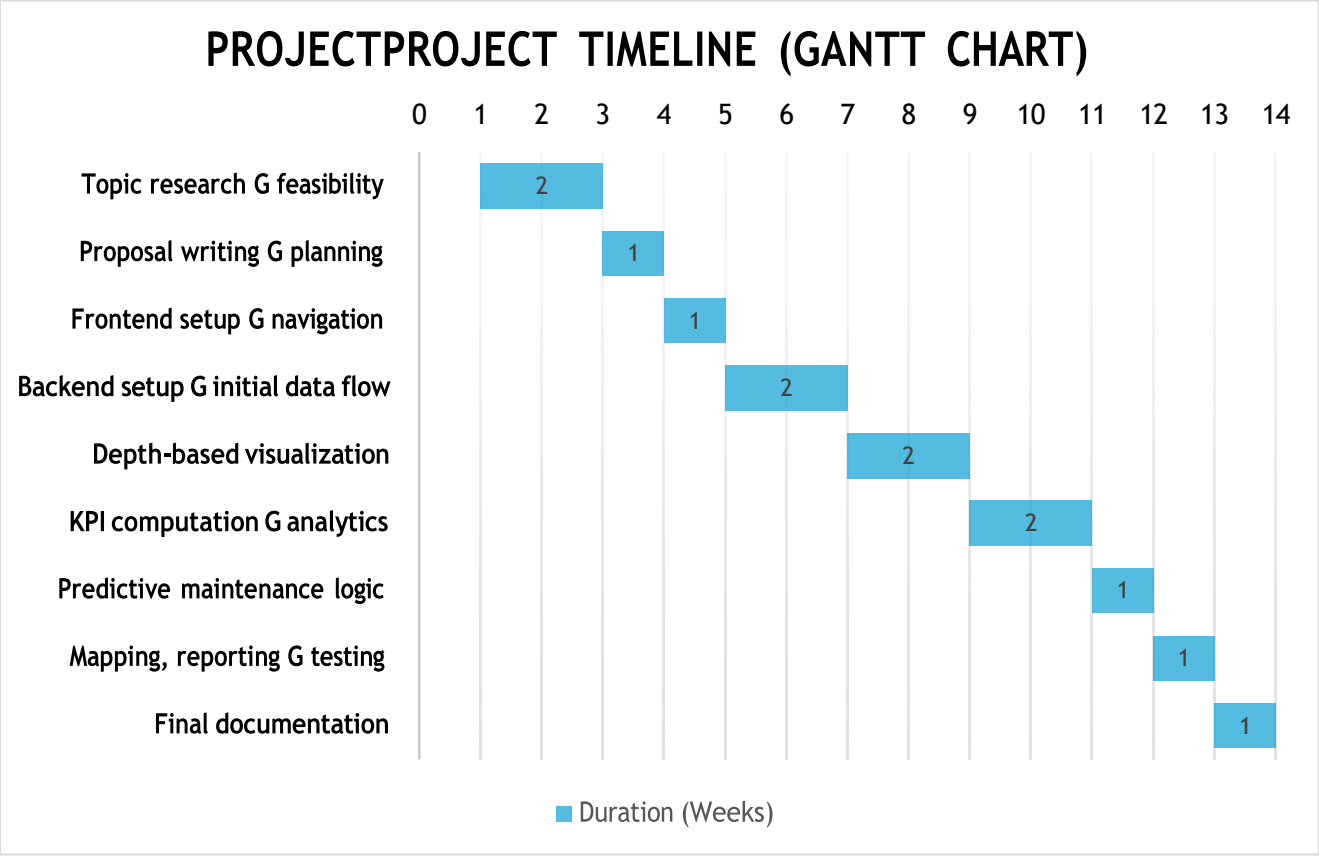
ChatGPT	GPT-5.2	Used as learning support while developing the React frontend, including understanding React components, JSX structure, routing with React Router, file organization, and troubleshooting common setup and import issues during implementation.	Supported faster understanding of React concepts, helped resolve configuration and import errors, and improved confidence in implementing the frontend correctly within the project timeline.
Figma AI	Free	Used to generate UI layout suggestions and design inspiration for the drilling dashboard, including card layout, spacing, and visual hierarchy.	Helped to clearly identify the existing research on the project and showed how the project differs from various working projects.
Copilot (Backend Development)	Free	Assisted with setting up the FastAPI backend, configuring SQLite, creating database models, organizing folder structure, and planning the data-upload workflow.	Accelerated backend setup, ensured clean architecture, and improved clarity on how data flows from upload → database → analytics → frontend.
Copilot (Backend Development)	Free	I worked through the Excel upload pipeline, including column normalization, validation, duplicate handling, row-level error reporting, and debugging the upload endpoint.	It helped me break down the errors clearly, understand what was causing the failures in the upload pipeline, and refine the normalization and validation flow so the backend is more stable, predictable, and better prepared for the analytics features I'll build next.

Project Planning and Timeline

Project Timeline

Phase	Week(s)	Milestones	Deliverables
Phase 1: Topic Exploration & Domain Research	Weeks 1 - 2	Explored potential project domains Reviewed drilling operations context Identified problem area and feasibility	Project idea selection Initial domain understanding
Phase 2: Proposal Development & Planning	Week 3	Defined project scope and objectives Identified data requirements Prepared a formal project proposal	Submitted project proposal Finalized research questions and methodology
Phase 3: Frontend Development (Core UI)	Week 4	Implement the main application layout Build well selection and navigation views	Initial React frontend Well selection interface
Phase 4: Backend Setup & Initial Data Flow	Weeks 5–6	Set up backend framework (FastAPI) Designed initial database schema Implemented basic API endpoints Connected frontend to backend using sample data	Functional backend service Initial API endpoints Frontend consuming backend data
Phase 5: Depth-Based Visualization	Weeks 7–8	Implemented vertical wellbore (pipe) visualization Mapped drilling events to depth intervals	Interactive depth-based visualization
Phase 6: KPI Computation & Analytics	Weeks 9–10	Compute NPT and event-based KPIs Aggregate data by depth and operation	KPI analytics module
Phase 7: Predictive Maintenance Logic	Weeks 11	Implement rule-based risk indicators Link risk to recorded events and equipment	Predictive maintenance indicators with explanations
Phase 8: Mapping & Report Generation & Testing	Weeks 12	Implement a well location map view Enable downloadable drilling summary reports. Test system functionality.	Location-based well view Report export feature Tested application
Phase 9: Final documentation	Week 13	Prepare final documentation	Final project report

Gantt chat – Project timeline



Appendix

AI Prompt(s) Used

- Explain React components, pages, layout, and routing like I'm a beginner, and guide me step-by-step to set up a React app using Vite on Windows.
- I'm building a drilling dashboard UI. Help me create a Week 4 React layout with wells list + navigation. Keep CSS in separate files and explain what each file does.
- I'm getting an import/casing error in Vite/React. Diagnose the error and tell me exactly what files to rename and what import paths to use.
- How do I modify the frontend such that when I click on the view detailed explanation, it goes ahead to show me the detailed explanation and the analytics gotten based on the data provided? Please also explain very well so I can understand how it is done.
- No there is already <button className="primaryBtn" type="button"> View Detailed Explanation </button> <button className="secondaryBtn" type="button" onClick={onClose}> Close </button> so when you chick on the button that's where the three pictures come in.
- Okay, when I put the backend it will generate the analytics on its own without putting any explanations
- I want to put a button to show back to reports after uploading it is successful. And also, the depth is not working again it did not capture the dept correctly. Also, i would like us to put a full designed upload on the report side because the current upload is when you click on the wells and go to view report. but this time i want it directly on the report page under the wells page in the component part.
- so my wellbore pipe segment is meant to show different color codes when it detects that the report that was uploaded has any critical word in it, in the description but it is not showing .

Microsoft Copilot

- let us work based on my project. do I need the data to start so that I can do some analytics to connect it to the front end? or let us start with , obviously you upload your dataset, then the app should receive this dataset and analyze it and show it on the wells page. When you then click on the particular well, it will show you the drilling pipe with different color coding. But first we should work on uploading the dataset first.
- For the uploading of files, these 3 categories, are you uploading the three documents as one csv, that is the; wells, operations, events. Also, most of this dataset have to be prepared after drilling right? that means there has to be some data input into their various tables. Or can the dataset from the company have everything, then the app begins to separate it by itself. That is to say, does it have to be prepared already.
- Good now, here in my wells, it looks so blank apparts from the wells that was created that is showing. What else do you think i can put here in the wells part to make it look more informative rather than showing only wells.
- when you click view report and upload report it shows page not found. can you link it to the exixting ones also move the quick bar to the side or draw a line to differenttiate it from the wells. Also i was thinking when you click on view reports you have to select the well first you want before seeing the reports. while upload report when you click on it, it can take us to the one we have already created.
- Okay now i see why the color codes are not showing on the segment. I tried editing one of the decription and i put pack-off inside so that it can display the color code. But i noticed the edit is not updated in the segment but it updated in the report details and backend. so the in the segment if you click the segment modal for each block it shows the original description but doesnt show the one i edited. so my testing did not work. Do you understand what i am trying to explain. so the segment is getting the description dirrectly from the uploaded report not minding if it was edited or not so it doesnt show my update. I edited it cause i just wanted to test the colors on the wellbore. Do you understand? and what can i do now?
- lets continue the upload i think most companies use excel not csv right so is it better to use excel to upload the data or csv based on what companies use.
- I dont want the exact names because not everyone in the industry would have time to put the exact names.
- okay if I integrate the script into the endpoint upload, that means it is ready for use. what

does that mean. Also, after integrating it, can i test it to see if it is working. secondly will the upload be able to carry thousands of roles of data.

- No i want it to show which report or date each NPT came from and put it in a bar chat. in the KPIModal for NPT. For example, the x axis will be the particular reportdate it occured, why the y axix will be the number of NPT. So when you click on the NPT KpiCard @Dashboards.jsx (95-108) , it goes to the modal and brings out the barchart of each report that has summed up to the the total NPT like i have described.
- I think creating the edit button in the report details so i can edit the mud and Equipment table if it did not work correctly. And also be able to save it after editing. I also noticed the mud details did not have the first column - Mud desc. please include it and put the edit table for the both.